5/9 Estimaciones Ágiles en Ingeniería y Calidad de Software (debrief)

Resumen ejecutivo

Este material sostiene una postura **pragmática** sobre estimaciones: **no** deben usarse como **compromisos** rígidos —eso "es peligroso y perjudicial"—, sino como una **práctica de aprendizaje** que **protege al equipo** y **verifica factibilidad** temprano. En **Ágil**, lo que se estima es el **tamaño relativo** de las historias (**Story Points**), **no tiempo**. La razón central es humana y práctica: **somos mejores comparando que adivinando horas**; la comparación **acelera** la dinámica grupal y **reduce** sesgos.

El **tamaño** es "la cantidad de trabajo" necesaria para una feature/story y **refleja** su **complejidad, trabajo requerido y magnitud**; por eso, **tamaño ≠ esfuerzo**. Las **estimaciones basadas en tiempo** fallan más por depender de **habilidades, conocimientos, supuestos, experiencia y familiaridad** con el dominio. Para medir tamaño, el equipo elige **una escala** (1–10, talles, 2^n o **Fibonacci**) y **no la cambia**: alterar la escala implica cambiar el **patrón de medida** y arruina la comparabilidad histórica.

Los **Story Points** son del **equipo** y combinan **complejidad, riesgo y esfuerzo** (en sentido relativo). La **Velocity** suma los SP **completos** por iteración (no parciales) y "**corrige los errores de estimación**" con evidencia empírica.

El método recomendado es **Planning Poker** (Cohn): definir una **story base canónica**, leer y aclarar con el PO, **votar en simultáneo**, discutir extremos y **repetir** hasta converger, **comparando siempre** con la canónica. El material brinda una **guía de interpretación**: 0 (no hay claridad), ½–1 (chica/cosmética), 2–3 (chica–mediana, *ideal*), 5 (media, *ideal*), 8 (grande, **dividir**), 13 (¿por qué no dividir?), 20 (¿justifica el negocio?), **40 (no entra en un sprint)**, **100 (no iniciar)**.

El cierre es enfático: **"Ágil es empírico"**; por lo tanto, **inspeccionar y adaptar es mandatorio**. Las estimaciones **sirven** si se tratan como **instrumentos de aprendizaje** y **planificación adaptativa**, no como promesas inamovibles. Con **tamaño relativo estable**, **Velocity** y **Planning Poker**, el equipo obtiene **previsibilidad práctica** sin perder la **capacidad de adaptación** que caracteriza al desarrollo ágil.

---

1. Introducción a las Estimaciones Ágiles

La estimación de software es una actividad compleja y fundamental en el desarrollo de productos. Por definición, una estimación no es precisa y la mayoría de las veces se cometen errores al estimar. Sin embargo, es un esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único, orientado a objetivos claros y alcanzables.

💡

Postura general sobre estimaciones

Pragmatismo, no dogma. “No ser dogmático sobre nada. Ser pragmático es la clave del éxito”.

Peligro de usarlas como compromisos. Si se usan como compromisos contractuales, “son muy peligrosas y perjudiciales”.

El valor está en el proceso. “Lo más beneficioso... es el *proceso de hacerlas*”.

Detección temprana y protección. Ayudan a **testear factibilidad** y “pueden servir como una gran protección para el equipo”.

1.1. ¿Qué es el Tamaño en Estimaciones Ágiles?

En el contexto ágil, el **tamaño** de una *feature* o *story* (característica o historia de usuario) indica:

- **Cuán compleja es**: Refleja la intrincada naturaleza de la funcionalidad.
- **Cuánto trabajo es requerido**: Se refiere a la cantidad bruta de esfuerzo necesario para su desarrollo.
- **Cuán grande es**: Una medida general de la magnitud de la funcionalidad.

El tamaño es una de las estimaciones más importantes, ya que su variación afecta drásticamente otros aspectos como el esfuerzo y el costo. En metodologías ágiles, el tamaño se refiere a la cantidad de trabajo necesaria para producir una *feature/story*.

1.2. Por qué las Estimaciones Basadas en Tiempo son Problemáticas

Las **estimaciones basadas en tiempo** son, por naturaleza, más propensas a errores debido a múltiples factores:

- **Habilidades y Conocimiento**: Las capacidades individuales del equipo influyen directamente en la velocidad y eficiencia.
- **Asunciones**: Las suposiciones hechas al inicio de un proyecto, a menudo con información incompleta, pueden llevar a desviaciones significativas.
- **Experiencia y Familiaridad con el Dominio**: La falta de experiencia previa con tecnologías o el dominio de negocio específico aumenta la incertidumbre y el riesgo de error en la estimación.
- **Contexto Único del Proyecto**: Cada proyecto, por similar que sea, posee características únicas (cronogramas, presupuestos, equipos, clientes, perfiles de riesgo) que dificultan la extrapolación directa de estimaciones de proyectos anteriores.

Siempre se busca conocer muy al inicio del proyecto el costo y el tiempo, pero la precisión es compleja debido al alcance aún no definido del producto.

1.3. Tamaño vs. Esfuerzo: Una Distinción Crucial

🚨

“Tamaño NO ES esfuerzo.”

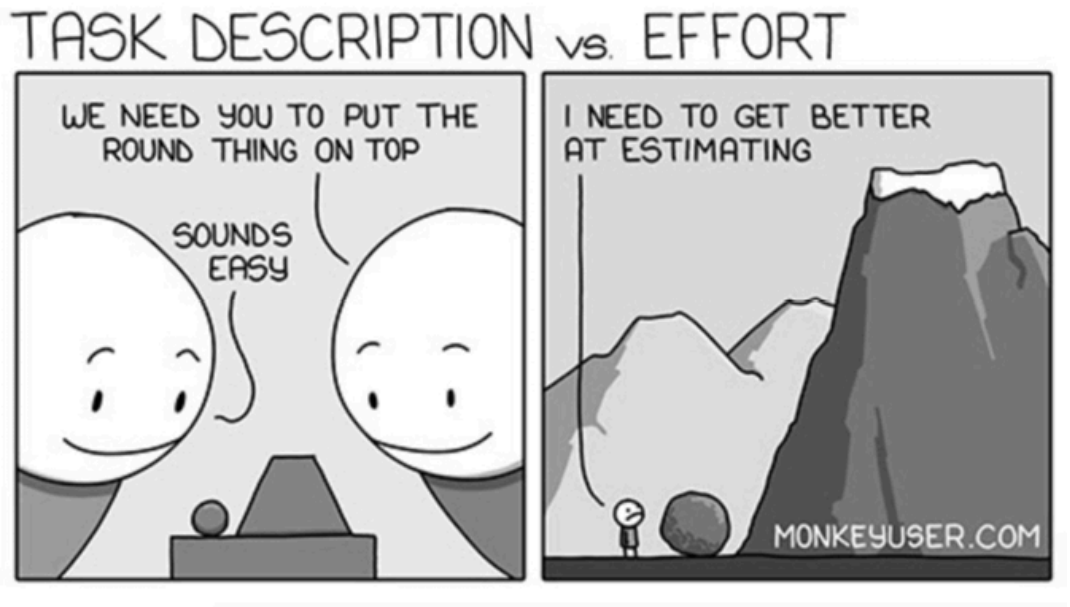
Las **estimaciones basadas en tiempo** son más propensas a error por **habilidades, conocimiento, supuestos, experiencia y familiaridad con el dominio**.

- El **tamaño** es una medida de la magnitud de la funcionalidad, a menudo relacionada con su complejidad e incertidumbre.

- El **esfuerzo** se refiere a las "horas persona" necesarias para construir el producto, es decir, el tiempo de trabajo puro sin considerar factores como reuniones, preparación de pruebas, revisiones, etc. Se mide en horas, persona, lineal.

El **esfuerzo está influenciado por el tamaño del software y la productividad del equipo**, pero también por el proceso de desarrollo, las características del proyecto, la organización, el equipo y el nivel de experiencia de las personas. Esto significa que dos historias con el mismo "tamaño" (ej. en Story Points) pueden requerir un esfuerzo muy diferente dependiendo de quién las implemente y el contexto.

2. El Concepto de Tamaño y su Medición en Entornos Ágiles



En metodologías ágiles, las estimaciones son un proceso y deben ser revisadas continuamente. La "mejor métrica de progreso es la cantidad de software funcionando". Son relativas a diferencia de las estimaciones profesionales.

Siempre es mas facil comparar de manera relativa que haciendo estimaciones absolutas, por ejemplo estimar un tamaño de un edificio es mas facil compararlo con otro edificio para así decir si es mas o menos grande que el otro, mientras que el enfoque tradicional estima de manera absoluta en este caso revelando la altura en formato numérico y no si es mas o menos grande que otro.

2.1. Story Points como Unidad de Medida

“Unidad de medida **específica del equipo** de **complejidad, riesgo y esfuerzo** (análoga al ‘kilo’ para peso)”.

Da idea del “**peso**” y decide cuán **grande/complex** es la story. La **complejidad tiende a crecer exponencialmente**.

Los Story Points consideran tres dimensiones principales para determinar el tamaño de una historia de usuario:

- **Complejidad**: Cuán compleja es la implementación de la historia (ej. número de partes y relaciones).
- **Esfuerzo**: Las horas persona lineales estimadas para construir la historia.
- **Incertidumbre**: Cuánta información falta para una estimación más precisa. Se refiere a riesgos técnicos o funcionales.

Qué se estima en Ágil: **tamaño**, no tiempo

- **Las features/stories se estiman en *Story Points* (SP)** como **medida de tamaño relativo**.
- **Relativo ≠ absoluto.** Los SP **no** son tiempo.
- **Por qué relativo:** "Las personas no saben estimar en términos absolutos; somos buenos **comparando**" (más rápido, mejora la dinámica de equipo).

2.1 A - VELOCIDAD.

Es una medida (métrica) del progreso de un equipo. Se calcula sumando el número de story points (asignados a cada user story) que el equipo completa durante la iteración.

- **Métrica de progreso**: suma de SP completados por el equipo en la iteración.
- **Solo se cuentan stories "completas"** (no parciales).
- La Velocidad corrige los errores de estimación.
- Es una metrica que se la mide al final del sprint en la entrega del incremento del producto (sprint review). Se calcula en base a los Story Points de las US aceptadas por el PO

🚨 **Velocity** para **proyectar** (y "corregir" estimaciones con evidencia).

2.2. Escala y Técnicas de Estimación de Tamaño

Dado que la complejidad del software crece exponencialmente, se adopta la **serie de Fibonacci** para la determinación de los Story Points (ej. 1, 2, 3, 5, 8, 13, 20, 40, 100). Esta escala no lineal evita una falsa sensación de exactitud en estimaciones grandes y fomenta la división de historias grandes en más pequeñas.

🚨 **Fibonacci (1,1,2,3,5,8,13,21,34,55,89,...)**.

Las técnicas más comunes para estimar el tamaño son:

Planning Poker (MAIN)

Es un método de juicio experto grupal, basado en el Wideband Delphi.

- **Proceso**: Cada miembro del equipo tiene una baraja de cartas con valores de Fibonacci. Para cada historia, se discute brevemente y luego cada miembro elige una carta con su estimación, colocándola boca abajo. Al mismo tiempo, se revelan las cartas. Si hay discrepancias significativas, el equipo discute las razones hasta alcanzar una estimación consensuada.
- **Participan desarrolladores** ("quienes resuelven las tareas son quienes deben estimarlas").

Se recomienda usar una canónica de tamaño 3 o 5, dando margen de maniobra para mayor o menor complejidad. Si cambio la canónica cambio la US que mide la comparación con el resto de las US por ende, compromete la estimación de todas las demás US, tendría que volver a recalcular todas las story points.

Prerrequisitos:

Lista de features/stories a ser estimadas y cada estimador tiene un mazo de cartas.

- **Secuencia explícita**:

1. Definir **story base canónica** (p. ej., “Story Z”).
2. Leer la story al equipo y **aclarar** con el PO.
3. Cada estimador **elige carta** (escala acordada) y la coloca boca abajo.
4. **Revelar en simultáneo**.
5. Si difieren, **discuten especialmente máximos y mínimos** y **repiten** (volver al paso 3).
6. Tomar la **siguiente** story y **comparar con la canónica** (“más/menos grande, compleja, riesgosa que Z?”).

Guía para “decodificar” tamaños SP

- **0**: no hay idea clara del producto/funcionalidad.
- **½–1**: funcionalidad chica (cosmética).
- **2–3**: funcionalidad chica a mediana (**lo que queremos**).
- **5**: funcionalidad media (**lo que queremos**).
- **8**: grande; **considerar dividir**.
- **13**: explicar por qué no puede dividirse.
- **20**: justificar razón de negocio y por qué no dividir.
- **40**: **no entra en un sprint**.
- **100**: algo está muy mal; **no iniciar**.

- **Beneficios**: Fomenta la colaboración, el entendimiento compartido del trabajo, y la rápida convergencia en las estimaciones. Permite que la persona que realiza el trabajo sea quien estima. El Product Owner participa, pero generalmente no estima, actuando como moderador y clarificador.

Wideband Delphi: Es un método grupal donde un grupo de personas se reúne para determinar el costo de desarrollo en esfuerzo y duración. Los expertos remiten sus estimaciones individuales de forma anónima, discuten las diferencias y revisan sus estimaciones hasta que convergen razonablemente.

Analogy: Se basa en la comparación de factores a estimar de forma relativa con proyectos anteriores similares. Sin embargo, en el agilismo, se argumenta que la experiencia no es directamente extrapolable a otros equipos o proyectos debido a sus particularidades.

Decomposition (Descomposición): Implica dividir el producto en unidades lo más pequeñas posible, estimando cada unidad y luego sumando para obtener una estimación total. En entornos ágiles, esto se aplica a las User Stories:

- **Epics (Épicas)**: Son *User Stories* grandes que no cumplen con los criterios INVEST (Independiente, Negociable, Valiosa, Estimable, Pequeña, Testeable) o no pueden ejecutarse en una iteración. Deben ser divididas en *User Stories* más pequeñas.
- **Themes (Temas)**: Agrupan *User Stories* relacionadas con un mismo tema.
- **Tareas**: Son las subdivisiones de una *User Story* en los pasos necesarios para implementarla. El Product Owner no suele preocuparse por las tareas, que son más de interés para el equipo de desarrollo.

2.3. Comparación con Métricas de Tamaño Tradicionales

En la gestión de proyectos tradicional, las métricas de tamaño suelen ser más formales y predictivas:

- **Líneas de Código (LDC)**: Históricamente usadas, pero actualmente en desuso debido a su dependencia del lenguaje de programación y la capacidad del programador.
- **Puntos de Función (PF)**: Una medida de la funcionalidad entregada al usuario, independiente de la tecnología.
- **Casos de Uso (CU)**: Otra métrica de tamaño que se relaciona con la complejidad de los casos de uso.

Estos métodos a menudo buscan una estimación "científica" y exhaustiva desde el principio del proyecto, a diferencia del enfoque iterativo y adaptativo del agilismo.

3. Descripción del Esfuerzo y su Relación con el Tamaño

El esfuerzo es un componente crítico en la estimación de proyectos, especialmente en un sector "intensivo en personas" como el desarrollo de software.

3.1. Definición de Esfuerzo

El **esfuerzo** se define como las "horas personas lineales" necesarias para construir el producto con la funcionalidad esperada por el cliente. Esto representa el tiempo de trabajo puro, sin considerar interrupciones o actividades no directamente relacionadas con la codificación o prueba de la funcionalidad.

3.2. Factores que Influyen en el Esfuerzo (más allá del tamaño)

Además del tamaño del software, el esfuerzo se ve afectado por una serie de factores:

- **Proceso de Desarrollo**: La eficiencia del proceso adoptado por el equipo.
- **Características del Proyecto**: Particularidades del proyecto en sí, como la complejidad técnica o los riesgos.
- **Organización y Equipo**: La estructura de la organización, la dinámica del equipo y la disponibilidad de recursos.
- **Personas**: El nivel de *expertise*, conocimiento de la tecnología, y familiaridad con el dominio de negocio de los miembros del equipo.
- **Productividad del Equipo**: La capacidad del equipo para convertir el esfuerzo en resultados.

Cuando no se dispone de datos históricos, se pueden usar tablas de productividad por tipos de software.

4.2. Momentos Clave para Estimar

Las estimaciones se realizan en momentos estratégicos del proyecto:

- Al inicio del proyecto.
- Luego de la especificación de requerimientos.
- Luego del diseño.

4.3. Riesgos en la Estimación

- **Incertidumbre inherente**: Por definición, una estimación no es precisa y existe un universo de probabilidades asociado a ellas. Los riesgos e incertidumbres son altos al inicio del proyecto.
- **Spikes**: Para mitigar la incertidumbre y el riesgo, se utilizan los "spikes", un tipo especial de *User Story* (invención de XP). Los *spikes* son tareas de investigación o prototipado con tiempo limitado, utilizadas para familiarizar al equipo con nuevas tecnologías o dominios, analizar el comportamiento de historias complejas para dividirlas, o ganar confianza frente a riesgos tecnológicos o funcionales. Son una excepción, no la regla, y deben usarse como última opción para incógnitas críticas. Una vez

resuelta la incertidumbre, el spike se convierte en una o más *User Stories*.
Suele pasar que cuando deseamos estimar una US y no sabemos cuanto ni como, esta US requiere de una spike para ser mas preciso.

4.4. Gestión de la Planificación en Múltiples Niveles (Enfoque Ágil)

Los equipos ágiles planifican al menos en tres niveles:

- **Planificación de Portfolio**: Priorización de diversos productos y decisión sobre cuáles continuar o descontinuar.
- **Planificación de Release**: Al inicio del proyecto, se determinan las épicas o *User Stories* que se incluirán en una nueva entrega (Release). Este plan se actualiza continuamente.
- **Planificación de Sprint/Iteración**: Se seleccionan las *User Stories* a implementar en la próxima iteración, se asignan estimaciones de esfuerzo y se desglosan en tareas. Esta es una reunión colaborativa entre el Product Owner y el equipo.

5. Principios Ágiles y Lean Aplicados a la Estimación y Planificación

Las estimaciones ágiles están profundamente arraigadas en la filosofía del Manifiesto Ágil y Lean.

5.1. Valores del Manifiesto Ágil y Estimación

Los valores y principios ágiles guían la forma en que se abordan las estimaciones.

- **Individuos e Interacciones sobre Procesos y Herramientas**: Se prioriza la comunicación fluida y la colaboración del equipo, lo que impacta directamente en la discusión y el consenso durante la estimación.
- **Software Funcionando sobre Documentación Detallada**: La entrega de software funcional es la mejor métrica de progreso, no la perfección de la estimación inicial. La documentación se genera cuando añade valor.
- **Colaboración con el Cliente sobre Negociación Contractual**: Implica estar dispuesto al cambio y cerca del cliente para predecir y ajustar las estimaciones y el alcance. Los requisitos pueden cambiar incluso en etapas finales.
- **Responder al Cambio sobre Seguir un Plan**: Los métodos ágiles son adaptables en lugar de predictivos. Las estimaciones no son promesas inquebrantables, sino puntos de partida que se refinan.

Un principio clave es la **simplicidad**, o "el arte de maximizar la cantidad de trabajo no realizado". Esto significa no agregar funcionalidades extras que el cliente no ha pedido, lo que impacta en la estimación de alcance.

5.3. Ciclos de Retroalimentación y Adaptación (Empirismo)

Tanto Scrum como Kanban se basan en el empirismo, donde la experiencia es la base para la formación de conocimiento. Los procesos definidos y la planificación detallada inicial son reemplazados por ciclos de inspección y adaptación.

- **Feedback Rápido**: Los ciclos de entrega cortos son esenciales para generar realimentación y adaptar el proceso rápidamente.
- **Inspección y Adaptación**: El equipo evalúa regularmente su desempeño y ajusta la forma de trabajar. El control se logra con inspecciones frecuentes y ajustes correspondientes.

🚨 **"Ágil es empírico; inspeccionar y adaptar es mandatorio."**

Antipatrones

- **Tratar SP como horas** → distorsiona el propósito de **tamaño relativo**.
- **Usar estimación como compromiso** → “peligroso y perjudicial”.
- **Cambiar escala/canónica** a mitad de camino → **rompe el “metro patrón”**